

Week 2

Application Layer

A complete deep-dive into network application protocols

2.1 Network Applications

2.2 Web & HTTP

2.3 Email Protocols

2.4 DNS

2.5 CDN

Prerequisites: Week 1 – Networking Basics & Information Security Awareness

2.1

Principles of Network Applications

Client-Server, P2P, Socket APIs & Application Architectures

2.1 — What is a Network Application?

Definition

A network application is software that communicates over a network using standard protocols. It runs on end systems (not routers) and exchanges messages.

Client-Server Architecture

Server: Always-on host, permanent IP, data centers

Client: Intermittent connection, dynamic IP, initiates requests

Examples: Web (HTTP), Email (SMTP), FTP

Real-world: When you open google.com, your browser (client) requests a page from Google's server

Peer-to-Peer (P2P) Architecture

No always-on server

Arbitrary end systems (peers) communicate directly

Self-scalable: each peer adds capacity

Examples: BitTorrent, Skype (historically), Blockchain

Real-world: Downloading a Linux ISO via BitTorrent — you download from many peers simultaneously

2.1 — Sockets, Transport Services & App Protocols



Socket = Door between application and transport layer

App → writes message to socket → OS delivers via TCP/UDP → arrives at destination socket → App reads it

TCP (Reliable)

- ✓ Reliable, in-order delivery
- ✓ Flow control & congestion control
- ✓ Connection-oriented (handshake)

Used by: HTTP, SMTP, FTP

Example: Loading a webpage — data must arrive complete and correct

UDP (Fast, Unreliable)

- ⚡ No connection setup overhead
- ⚡ No reliability guarantee
- ⚡ Faster, lower latency

Used by: DNS, Video streaming

Example: Live video call — speed > perfection

2.2

Web and HTTP

HyperText Transfer Protocol — The Foundation of the World Wide Web

2.2 — HTTP: HyperText Transfer Protocol

HTTP is a request-response protocol over TCP (port 80 / HTTPS port 443). It is stateless — server maintains no info about past requests.



HTTP Methods

GET	POST	PUT	DELETE
Retrieve a resource (load webpage)	Submit data (login form, upload)	Update/replace a resource	Remove a resource

Common HTTP Status Codes:

200 OK	301 Redirect	404 Not Found	500 Server Error
--------	--------------	---------------	------------------

Real-world Example: Type 'https://www.google.com' → browser sends GET / HTTP/1.1 Host: www.google.com → server replies 200 OK + HTML page

2.2 — Persistent vs Non-Persistent HTTP & Cookies

Non-Persistent HTTP (HTTP/1.0)

- New TCP connection per object
- 1 page + 10 images = 11 connections!
- 2 RTTs per object (SYN + request)
- OS overhead for each connection

Example: Fetching a page with 3 images: TCP connect → GET page → close → TCP connect → GET img1 → close...

Persistent HTTP (HTTP/1.1 & HTTP/2)

- Single TCP connection for all objects
- Pipelining: send multiple requests without waiting
- HTTP/2: multiplexing, header compression
- Much faster page loads

Example: Modern browser opens 1 connection and streams HTML + CSS + JS + images simultaneously

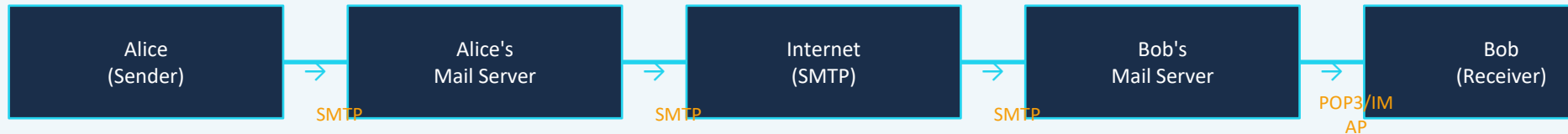
 **Cookies:** Since HTTP is stateless, cookies maintain state across requests. Server sends Set-Cookie header → browser stores it → sends it back with every future request to that domain. Used for: login sessions, shopping carts, user preferences.

2.3

Electronic Mail Protocols

SMTP · POP3 · IMAP — How Email Works Under the Hood

2.3 — Email System Architecture & SMTP



SMTP (Port 25 / 587)

- Push protocol — sender initiates
- Uses TCP for reliable delivery
- Commands: HELO, MAIL FROM, RCPT TO, DATA, QUIT
- Messages in 7-bit ASCII
- MIME extensions for attachments & HTML

Example dialogue:

```
S: HELO mail.alice.com
S: MAIL FROM: alice@example.com
S: RCPT TO: bob@example.com
S: DATA → message body → .
```

POP3 vs IMAP (Access Protocols)

POP3 (Port 110):

- Downloads mail to local device
- Deletes from server (by default)
- Simple: authorize → transaction → update
- Good for: single device use

IMAP (Port 143):

- Keeps mail on server
- Syncs across multiple devices
- Folders, flags, search on server
- Good for: phone + laptop + web access

Example: Gmail uses IMAP — read on phone, same inbox on laptop

2.4

DNS — Domain Name System

The Internet's Distributed Phone Book

2.4 — DNS: How Domain Names Get Resolved

DNS translates human-readable names (www.google.com) into IP addresses (142.250.195.46). It's a distributed, hierarchical database using UDP port 53.

DNS Hierarchy

Root DNS Servers (13 sets globally)

TLD Servers (.com, .org, .in, .edu)

Authoritative Servers (google.com, iitm.ac.in)

Local DNS (your ISP or 8.8.8.8)

DNS Record Types

Type	Meaning	Example
A	IPv4 address	google.com → 142.250.x.x
AAAA	IPv6 address	google.com → 2607:f8b0:....
MX	Mail server	gmail.com → smtp.gmail.com

Resolution Example

You type: **www.amazon.com**

1. Check local DNS cache
2. Ask local resolver
3. Root server → '.com' TLD
4. TLD → amazon.com auth. server
5. Auth. server → IP address
6. Browser connects to IP!

2.5

Content Distribution Networks

Bringing the Internet Closer to You

2.5 — Content Distribution Networks (CDNs)

Problem: A single origin server in the US serving billions of global users causes high latency and bottlenecks. CDNs solve this by distributing content to servers worldwide.

✗ Without CDN

- Single server in one location
- High latency for distant users
- Server overload during traffic spikes
- Single point of failure

Example: Netflix server in USA → **user in Chennai waits 200ms+**

✓ With CDN

- Hundreds of edge servers worldwide
- User served from nearest server
- Load distributed globally
- Redundancy & high availability

Example: Netflix CDN edge in Mumbai → **Chennai user gets <20ms latency!**

How CDN Works: User requests video.mp4 → DNS redirects to nearest CDN edge → Edge checks cache → If cached: serve immediately | If not: fetch from origin, cache, then serve

Major CDN Providers:

Akamai

Cloudflare

AWS CloudFront

Fastly

Netflix Open Connect

Week 2 Summary — Application Layer Protocol Comparison

Protocol	Transport	Port	Purpose	Real Example
HTTP/HTTPS	TCP	80 / 443	Web page transfer	Loading google.com in browser
SMTP	TCP	25 / 587	Sending email	Sending email via Gmail
POP3	TCP	110	Receiving email (download)	Outlook downloading inbox
IMAP	TCP	143	Receiving email (sync)	Gmail on phone + laptop
DNS	UDP	53	Name → IP resolution	Typing google.com → IP lookup
BitTorrent	TCP/UDP	6881+	P2P file sharing	Downloading Linux ISO

Key Insight: All application layer protocols run on end systems, not the network core. They rely on TCP (reliable) or UDP (fast) for transport. DNS is the glue that makes names work. CDNs make the web fast globally.

Week 2 Complete!

Application Layer — Topics Covered

- 1 2.1 Principles of Network Applications (Client-Server, P2P, Sockets, TCP vs UDP)
- 2 2.2 Web & HTTP (Request/Response, Methods, Persistent Connections, Cookies)
- 3 2.3 Email Protocols (SMTP for sending, POP3 & IMAP for receiving)
- 4 2.4 DNS (Hierarchical resolution, Record types, Caching)
- 5 2.5 Content Distribution Networks (Edge caching, CDN providers)

Next: Week 3 — Transport Layer (TCP, UDP, Congestion Control)